

Introduction to Statistical Signal Processing

Course Project

Project Members

Member 1

Name: Ashlin K Sunil

Roll Number: 2024102025

Department: Electronics and Communication Engineering (ECE)

Email ID: ashlin.sunil@students.iiit.ac.in

Member 2

Name: Bejaine C Birju

Roll Number: 2024102060

Department: Electronics and Communication Engineering (ECE)

Email ID: bejaine.birju@students.iiit.ac.in

Member 3

Name: Sanshray Vinesh

Roll Number: 2024102071

Department: Electronics and Communication Engineering (ECE)

Email ID: sanshray.vinesh@students.iiit.ac.in

Contents

Question 1	4
Question	4
Aim	4
Theory	4
Questions asked	5
a) Qualitative Effect of the Parameter k_0 (Setting $n_0 = k_0 + 1$)	5
b) Qualitative Effect of Filter Order ($p = 2$ to $p = 10$)	5
c) Which of the two signals is easier to denoise?	5
Code	6
Plots	8
Question 2	11
Aim	11
Theory	11
Code explanation	11
Results	14
Questions asked	15
a) Optimum value of the parameter λ for this task	15
Question 3	16
Question	16
Aim	16
Theory	16
Code explanation	17
Plots	20
Questions asked	21
a) Compute compression gain as a function of filter length and it's use to identify optimal filter length	21
b) Effect of changing value of M and N in affecting the performance	21

Question 1

Question

You are given two ECG signals corrupted with noise (Signal1a & Signal1b). You can assume that the original ECG signals (which are not directly available) are noise-free and stationary. For each of these signals, perform noise cancellation without any reference. The method is discussed in section 9.2.5 of the textbook.

- Qualitatively investigate the effect of the parameter k_0 ; set $n_0 = k_0 + 1$
- Qualitatively investigate the performance as the filter order is changed from $p = 2$ to $p = 10$
- Which of the two signals is easier to denoise? Comment on possible reasons for this

Aim

We aim to implement reference free noise cancellation on two different signals and see the effect of delay (k_0) and filter order (p) on the effectiveness of the filter.

Theory

In a standard adaptive noise canceller, you need a primary noisy signal $x(n) = d(n) + v_1(n)$ and a separate reference signal $v_2(n)$ that is correlated with the noise but uncorrelated with the desired signal $d(n)$.

When a secondary reference is unavailable, we can artificially generate one by delaying the primary signal itself, creating the reference $x(n - n_0)$. This approach relies entirely on the contrasting autocorrelation properties of the two signal components:

- The Signal ($d(n)$) : An ECG is a narrowband or periodic process. It has a long correlation length, meaning $d(n)$ remains highly correlated with delayed versions of itself, $d(n - n_0)$.
- The Noise ($v_1(n)$) : The corrupting noise is a broadband process. and it has short correlation length, meaning it completely loses correlation with itself after a specific number of samples k_0 .
Mathematically, $E\{v_1(n)v_1(n - k)\} = 0$ for $|k| > k_0$.

If we set the delay $n_0 > k_0$, the cross-correlation between the current noise and the delayed observation becomes zero :

$$E\{v_1(n)x(n - k)\} = E\{v_1(n)d(n - k)\} + E\{v_1(n)v_1(n - k)\} = 0$$

Because the adaptive filter can only predict and pass components that are correlated between its input and its reference, it becomes “blind” to the decorrelated noise and exclusively predicts the correlated ECG signal $d(n)$.

In the standard LMS algorithm, the step size (the speed at which the filter learns) is a fixed value. If the input signal power is very high, the algorithm can become unstable and diverge; if the power is very low, it learns too slowly. Normalized Least Mean Squares (NLMS) is an enhanced version of the

standard LMS adaptive filter designed to address the sensitivity of the algorithm to the power of the input signal.

Unlike LMS, NLMS uses a time-varying step size. It calculates the input power at each step and adjusts the update accordingly and because it adapts its learning rate to the signal strength, it often converges much faster than standard LMS, especially when dealing with non-stationary signals

Questions asked

a) Qualitative Effect of the Parameter k_0 (Setting $n_0 = k_0 + 1$)

The parameter k_0 represents the absolute boundary of the noise's "memory" or correlation length. The choice of the delay parameter n_0 directly dictates the success of the filter:

- When $n_0 < k_0$ (Insufficient Delay) : The delayed noise $v_1(n - n_0)$ is still statistically correlated with the current noise $v_1(n)$. Because the adaptive filter is designed to pass correlated components, it will predict and pass this noise through to the output $\hat{d}(n)$. The resulting ECG estimate will remain heavily corrupted.
- When $n_0 > k_0$ (Optimal Delay) : The delayed noise becomes perfectly uncorrelated with the current noise. The adaptive filter will stop predicting the noise entirely. The only remaining correlated component between $x(n)$ and $x(n - n_0)$ is the ECG signal, allowing the filter to converge to a clean estimate.
- When $n_0 \gg k_0$ (Excessive Delay) : While ECG signals are periodic, they are technically quasi-periodic due to natural heart rate variability (HRV). If the delay n_0 is set far too high, the ECG signal $d(n)$ will eventually start to lose correlation with $d(n - n_0)$. This causes the filter to struggle to predict the ECG, leading to signal attenuation or phase distortion.

b) Qualitative Effect of Filter Order ($p = 2$ to $p = 10$)

The filter order p represents the number of tap weights (degrees of freedom) the LMS algorithm has to shape its frequency response and reconstruct the signal.

- Low Order ($p = 2$) : A 2nd-order filter only has two degrees of freedom, which restricts it to modeling a single, simple sinusoidal component (one narrow peak in the frequency domain). An ECG signal, however, is a complex waveform (particularly the sharp QRS complex) made up of many fundamental harmonics. A $p = 2$ filter will severely low-pass or distort the ECG, smoothing out the R-peaks and destroying the morphological features of the wave.
- High Order ($p = 10$) : Increasing the order gives the adaptive filter the ability to form a complex, multi-band frequency response. It can place poles/zeros to accurately track the multiple harmonics required to reconstruct the sharp, transient spikes of the QRS complex without excessive smoothing, resulting in a much truer reconstruction of the original ECG.

c) Which of the two signals is easier to denoise?

The signal corrupted by "whiter" noise (noise with a smaller k_0) is significantly easier to denoise. This is because if the noise is nearly white, k_0 approaches 0, which means you only need a very small delay (e.g., $n_0 = 1$ or 2) to perfectly decorrelate the noise. At such a small delay, the ECG signal $d(n)$ is still near-perfectly correlated with $d(n - n_0)$, making the adaptive filter's job trivial.

Conversely, if the noise is highly “colored” (e.g., a low-frequency drift or a heavily filtered autoregressive noise process), k_0 is very large. To decorrelate this noise, you are forced to use a massive delay n_0 . Because of the heart’s natural beat-to-beat variability, forcing the filter to predict the current ECG spike using an observation from hundreds of samples ago degrades the signal tracking. Therefore, colored noise forces a difficult trade-off between noise rejection and ECG distortion, making it much harder to clean.

Code

```
%% Problem 1

% Noise cancellation of ECG signals without reference signal and
% qualitative analysis of  $k_0$  and  $p$  through NLMS
clear; clc; close all;

load('Signal1a.mat'); % y_new_2
load('Signal1b.mat'); % y_new_1

data_list = {y_new_2, y_new_1};
signal_labels = {'Signal 1a', 'Signal 1b'};

%  $k_0$  and  $p$  values
k0_vals = [2, 10, 50];
p_fixed = 5;

p_vals = [2, 5, 10];
k0_fixed = 10;

% Normalized step size (beta) since we are using NLMS
beta = 0.1;
ignore_samples = 200; % Samples to skip in plots to hide filter convergence

% Variables to store results for Part C comparison
best_y_1a = [];
best_y_1b = [];

for s = 1:2
    x = data_list{s}(:); % Ensuring signal is a column vector

    %% Figure 1: Effect of  $k_0$  (with a fixed  $p$ )

    figure('Name', [signal_labels{s}, ' - Effect of  $k_0$ '], 'Position', [100, 100, 800,
600]);
    for i = 1:length(k0_vals)
        k0 = k0_vals(i);
        n0 = k0 + 1;

        % Running NLMS Adaptive Line Enhancer
        [y_out, ~] = nlms_ale(x, p_fixed, n0, beta);

        subplot(3, 1, i);
```

```

% Plot of skipping the convergence transient
n_plot = ignore_samples:length(x);
plot(n_plot, x(n_plot), 'Color', [0.8 0.8 0.8]); hold on;
plot(n_plot, y_out(n_plot), 'r', 'LineWidth', 1.2);

title(sprintf('%s | k_0 = %d (Fixed p = %d)', signal_labels{s}, k0, p_fixed));
xlabel('Samples'); ylabel('Amplitude');
xlim([500 1500]);
legend('Original Noisy', 'Denoised', 'Location', 'northeast');
grid on;
end

%% Figure 2: Effect of Filter Order p (with a fixed k0)

figure('Name', [signal_labels{s}, ' - Effect of p'], 'Position', [150, 150, 800,
600]);
for j = 1:length(p_vals)
    p = p_vals(j);
    n0 = k0_fixed + 1;

    % Running NLMS Adaptive Line Enhancer
    [y_out, ~] = nlms_ale(x, p, n0, beta);

    % Save the best result (p=10, k0=10) for Part C
    if p == 10
        if s == 1, best_y_1a = y_out; else, best_y_1b = y_out; end
    end

    subplot(3, 1, j);
    plot(n_plot, x(n_plot), 'Color', [0.8 0.8 0.8]); hold on;
    plot(n_plot, y_out(n_plot), 'b', 'LineWidth', 1.2);

    title(sprintf('%s | p = %d (Fixed k_0 = %d)', signal_labels{s}, p, k0_fixed));
    xlabel('Samples'); ylabel('Amplitude');
    xlim([500 1500]);
    legend('Original Noisy', 'Denoised', 'Location', 'northeast');
    grid on;
end
end

%% Part C - Comparison of Signal 1a vs 1b
figure('Name', 'Part C: Denoising Comparison', 'Position', [200, 200, 800, 400]);

subplot(2,1,1);
plot(n_plot, best_y_1a(n_plot), 'r', 'LineWidth', 1.2);
title('Best Denoised Signal 1a (p=10, k_0=10)');
xlim([500 1500]); grid on;

subplot(2,1,2);
plot(n_plot, best_y_1b(n_plot), 'b', 'LineWidth', 1.2);
title('Best Denoised Signal 1b (p=10, k_0=10)');
xlim([500 1500]); grid on;

```

```

sgtitle('Part C: Which signal is easier to denoise?');

%% Function: Normalized LMS Adaptive Line Enhancer (NLMS-ALE)

function [y, e] = nlms_ale(x, p, n0, beta)
    N = length(x);
    y = zeros(N, 1);
    e = zeros(N, 1);
    w = zeros(p, 1);
    epsilon = 1e-6; % Small constant to prevent division by zero

    for n = (p + n0):N
        % u is the delayed signal (reference input)
        u = x(n-n0 : -1 : n-n0-p+1);

        y(n) = w' * u; % Prediction of the clean signal
        e(n) = x(n) - y(n); % Prediction error (Noise)

        % NLMS Weight Update (Divided by u'*u)
        norm_u = u' * u;
        w = w + (beta / (norm_u + epsilon)) * e(n) * u;
    end
end

```

Plots

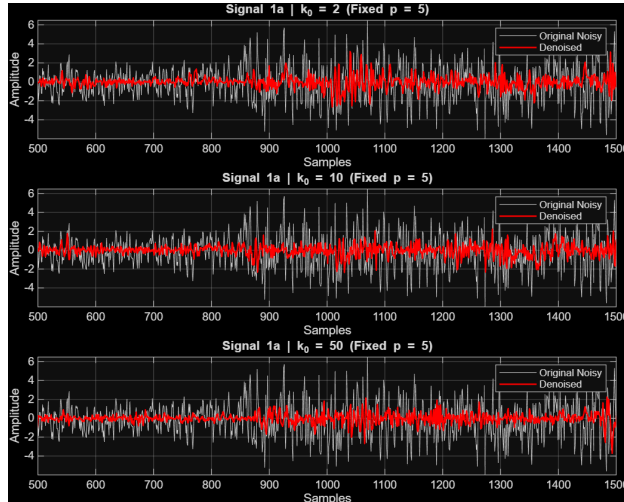


Figure 1: Effect of K0

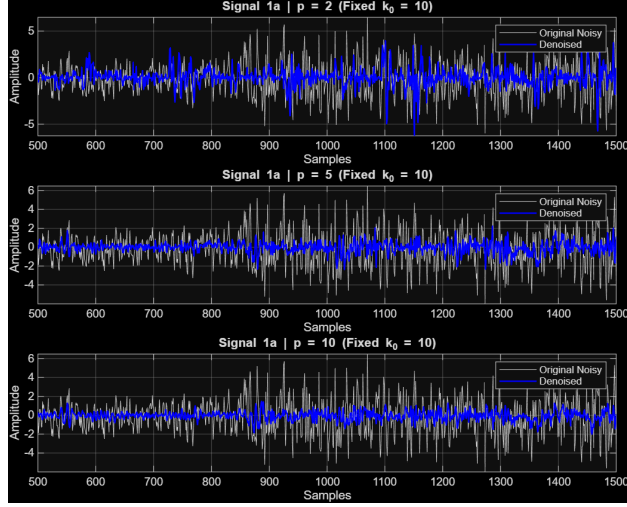


Figure 2: Effect of p

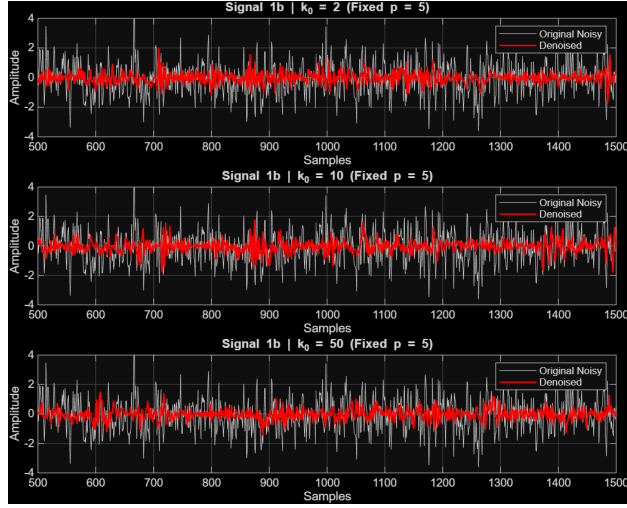


Figure 3: Effect of k_0

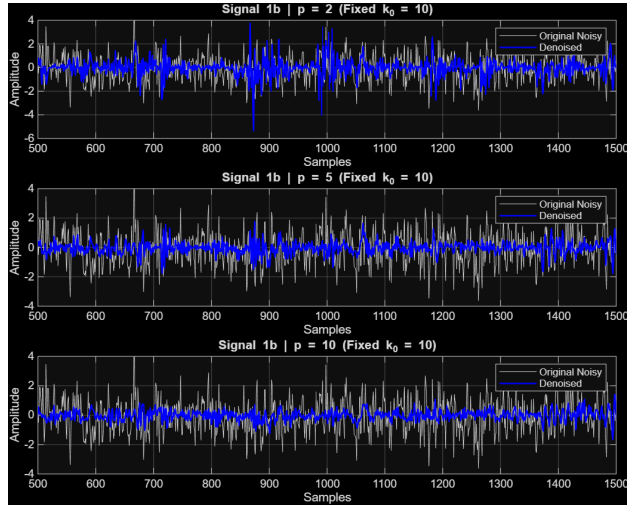


Figure 4: Effect of p

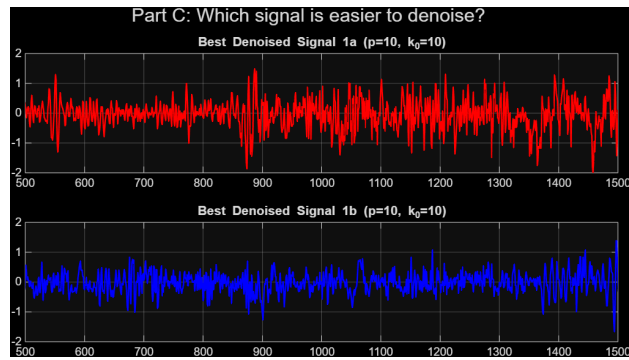


Figure 5: Denoising Comparison

Question 2

Aim

We aim to approximate an unknown system which has both the input and required outputs given. It will be implemented both as SDAF and RLS adaptive filter. The effect of forgetting factor and filter order will also be quantified

Theory

System identification involves finding a mathematical model (an LTI system) that maps an input $x[n]$ to an output $y[n]$. The Steepest Descent Adaptive Filter (SDAF) assumes the signals are Wide-Sense Stationary (WSS), meaning their first and second moments (mean and correlation) do not change over time. Ergodicity allows us to estimate these statistical moments (the autocorrelation matrix R and cross-correlation vector P) using time averages from a single realization of the signal. The optimal LTI filter weights w_{opt} that minimize the Mean Square Error (MSE) are defined by $R * w_{\text{opt}} = P$

Unlike gradient-based methods, RLS aims to minimize a weighted least squares cost function that accounts for all previous data. The scalar λ (typically $0.9 < \lambda < 1$) determines the algorithm's memory. $\lambda = 1$ gives all past data equal weight (infinite memory), while $\lambda < 1$ exponentially discounts older samples.

Code explanation

```
%% Problem 2
clear; clc; close all;

% Load signal data into vectors
load('Signal2.mat');
xn = xn(:); yn = yn(:);
N = length(xn);

%% Part A - SDAF (Steepest Descent Adaptive Filter)
p_const = 5;

% Estimate Autocorrelation (R) and Cross-correlation (P) assuming WSS/Ergodicity
r_full = xcorr(xn, p_const-1, 'biased');
R = toeplitz(r_full(p_const:end));
p_full = xcorr(yn, xn, p_const-1, 'biased');
P_vec = p_full(p_const:end);
P_vec = P_vec(:);

% 10% of the maximum mu limit for stability
mu = 0.1 * (2 / max(eig(R)));

w_sdaf = zeros(p_const, 1);
iterations = 1000;
for i = 1:iterations
```

```

    w_sdaf = w_sdaf + mu * (P_vec - R * w_sdaf);
end

%% Part B - RLS Lambda Analysis & Comparison
% A smaller lambda (0.9) adapts fast but "shuffles" (more noise).
% A larger lambda (1.0) adapts slow but is smoother in steady-state.
lambdas = [0.9, 0.95, 0.99, 1.0];
delta = 1.0;
mse_comparison = zeros(length(lambdas), 1);

figure; hold on;
for i = 1:length(lambdas)
    l = lambdas(i);
    w = zeros(p_const, 1);
    P_mat = eye(p_const) / delta;
    e_plot = zeros(N, 1);

    for n = p_const:N
        u = xn(n:-1:n-p_const+1);

        % Gain vector update with stabilizer to prevent matrix explosion
        denom = l + u' * P_mat * u + 1e-6;
        k = (P_mat * u) / denom;

        e_plot(n) = yn(n) - w' * u;
        w = w + k * e_plot(n);
        P_mat = (P_mat - k * u' * P_mat) / l;
    end

    % Plotting raw squared error to see the "shuffling" noise
    plot(e_plot.^2, 'DisplayName', ['\lambda = ', num2str(l)]);

    % Calculate MSE using second half of the signal for steady-state analysis
    mse_comparison(i) = mean(e_plot(round(N/2):end).^2);
end

title('Part (b): RLS Learning Curves (Linear Scale)');
xlabel('Time index n'); ylabel('Squared Error (e^2)');
legend; grid on;

ylim([0 0.015]);

% Selection of the best parameter based on steady-state MSE
[~, best_idx] = min(mse_comparison);
opt_lambda = lambdas(best_idx);
fprintf('Optimum lambda selected: %.2f\n', opt_lambda);

%% Part C - Plot Filter Coefficients for Selected Lambda and p=5
% Re-run RLS with the selected optimum lambda to track weights
w_rls = zeros(p_const, 1);
P_mat = eye(p_const) / delta;
w_history = zeros(p_const, N);

```

```

e_rls = zeros(N, 1);

for n = p_const:N
    u = xn(n:-1:n-p_const+1);
    k = (P_mat * u) / (opt_lambda + u' * P_mat * u + 1e-6);
    e_rls(n) = yn(n) - w_rls' * u;
    w_rls = w_rls + k * e_rls(n);
    P_mat = (P_mat - k * u' * P_mat) / opt_lambda;
    w_history(:, n) = w_rls;
end

figure;
plot(w_history');
title(['Part (c): RLS Coefficient Convergence (\lambda = ', num2str(opt_lambda), ')]);
xlabel('Time index n'); ylabel('Coefficient Values');
legend('w1', 'w2', 'w3', 'w4', 'w5'); grid on;

%% Part D - System Approximation Accuracy
% Plotting the difference between given output y[n] and system output.
figure;
plot(e_rls);
title('Part (d): Instantaneous Accuracy (y[n] - \hat{y}[n])');
xlabel('Time index n'); ylabel('Error'); grid on;

%% Part E- Overall Accuracy vs. Filter Length (p = 1 to 10)
% Analysis of system approximation accuracy as a function of the filter order p.
p_range = 1:10;
mse_results = zeros(length(p_range), 1);

for idx = 1:length(p_range)
    p_curr = p_range(idx);
    w_tmp = zeros(p_curr, 1);
    P_tmp = eye(p_curr) / delta;
    e_tmp = zeros(N, 1);

    for n = p_curr:N
        u_tmp = xn(n:-1:n-p_curr+1);
        % Gain update with numerical stabilizer
        k_tmp = (P_tmp * u_tmp) / (opt_lambda + u_tmp' * P_tmp * u_tmp + 1e-6);
        err = yn(n) - (w_tmp' * u_tmp);
        e_tmp(n) = err;
        w_tmp = w_tmp + k_tmp * err;
        P_tmp = (P_tmp - k_tmp * u_tmp' * P_tmp) / opt_lambda;
    end
    mse_results(idx) = mean(e_tmp(round(N/2):end).^2);
end

figure;
plot(p_range, mse_results, 'r-o', 'LineWidth', 1.5);
title('Part (e): Steady-State Accuracy vs. Filter Order p');
xlabel('Filter Order p'); ylabel('Overall MSE'); grid on;

```

Results

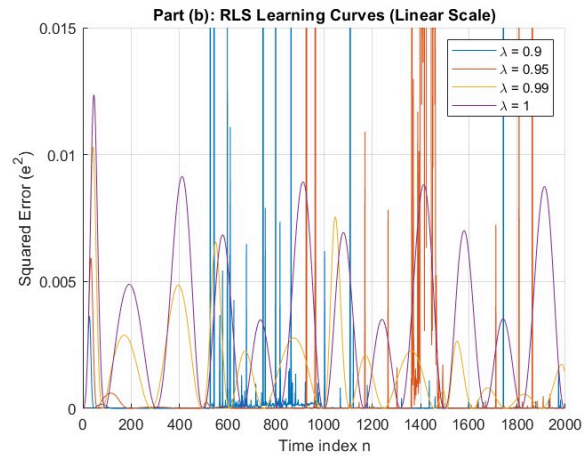


Figure 6: Learning curves for different λ values

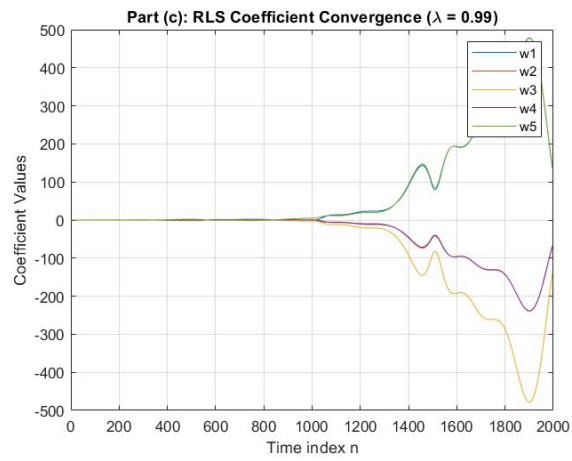


Figure 7: Coefficient values change over times with fixed λ

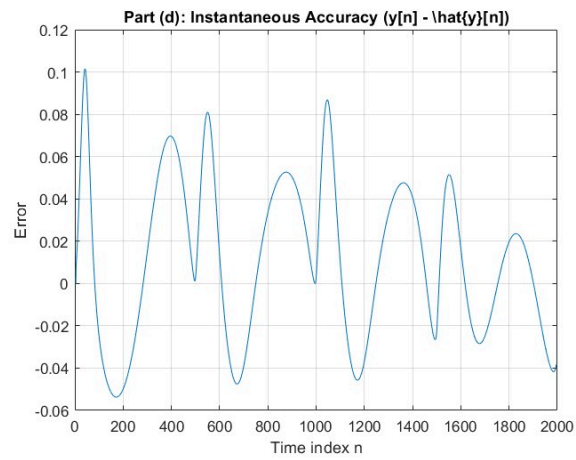


Figure 8: Instantaneous accuracy of output

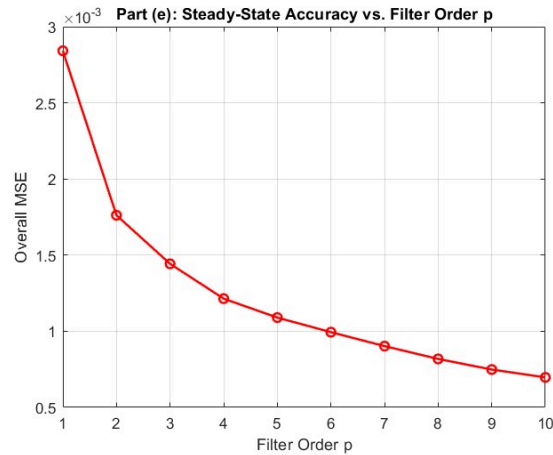


Figure 9: Accuracy with changing filter order

Questions asked

a) Optimum value of the parameter λ for this task

The parameter λ , called the forgetting factor, weighs the decay of older data samples. In general, finding the optimal value of λ is a balancing act between two competing factors: convergence speed and steady-state stability. To find the optimal value, you must evaluate your specific signal environment, weighing the factors as given below. For the given question, we just test a few different lambda values and see which one has the best performance

Feature	Low λ (e.g., 0.95)	High λ (e.g., 0.999)
Tracking Ability	Fast; follows non-stationary changes quickly.	Slow; struggles to follow rapid system changes.
Noise Rejection	Poor; the filter is “jittery” and sensitive to noise.	Excellent; provides smooth, averaged estimates.
Stability	Higher risk of the covariance matrix becoming unstable.	Generally more robust and stable.

Question 3

Question

An audio signal is given to you (file Signal3). It is known that an auto-regressive model is a good approximation for this signal. Divide the signal into $M = 5$ equal portions. For each portion, use the first $N = 10\%$ to learn the AR model parameters using an LMS filter and apply it to compress the rest of the 90% of the signal. Define compression gain to be the ratio of signal norm and error norm.

- Compute and plot the (average) compression gain as a function of filter length. How would you use this to identify optimal filter length.
- Explain using simulations – if and how any changes (increase or decrease) in M and N will affect the performance.

Aim

We aim to learn AR model parameters which were used to generate a specific signal using a LMS filter and then use this to compress the signal. Effect of filter length, splitting the sample regions and increasing width of sample regions will also be quantified.

Theory

An auto-regressive (AR) process of order p models a signal sample $x[n]$ as a linear combination of its past p samples plus a white noise term $w[n]$. In audio signals, this model assumes that the current sound sample is highly correlated with previous ones due to the physical properties of the sound source.

$$x[n] = \sum_{i=1}^p a_i * x[n-i] + w[n]$$

To estimate the AR coefficients (a_i) in real-time without solving complex matrix equations (like the Wiener–Hopf equations), adaptive filtering is used. The filter generates an estimate $\hat{x}[n]$, where $\mathbf{u}[n]$ is the vector of past samples:

$$\hat{x}[n] = \mathbf{w}[n]^T * \mathbf{u}[n]$$

The coefficients are updated at each step using the prediction error:

$$e[n] = x[n] - \hat{x}[n]$$

Because audio signals undergo severe amplitude fluctuations, the Normalized LMS (NLMS) algorithm is utilized to guarantee stability. NLMS scales the step-size μ by the instantaneous power of the input vector:

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \left(\frac{\mu}{\mathbf{u}[n]^T * \mathbf{u}[n] + \varepsilon} \right) * e[n] * \mathbf{u}[n]$$

Compression (Predictive Coding): In this system, compression is achieved through predictive coding. Instead of storing or transmitting the original audio signal $x[n]$ (which has a large dynamic range), we only store the fixed filter weights w and the prediction error (residual) $e[n]$. If the AR model accurately predicts the signal, the residual $e[n]$ will have a much smaller variance and norm than the original signal.

This efficiency is quantified by the compression gain:

$$G = \frac{|x_{\text{test}}|_2}{|e_{\text{test}}|_2}$$

A higher G indicates that the error norm is significantly smaller than the signal norm, yielding highly efficient compression.

Code explanation

```
% Problem 3
% AR Compression Analysis using NLMS

clear; clc; close all;

filename = 'Signal3.wav';

[x, Fs] = audioread(filename);

% Converting to mono if stereo
if size(x, 2) > 1
    x = mean(x, 2);
end

% Ensuring x is a column vector
x = x(:);

%% Part (a) : Compute and plot the (average) compression gain as a function of filter
length.

M = 5; % Number of equal portions
N_pct = 0.1; % N = 10% for training
p_values = [2, 5, 10, 20, 30, 50, 75, 100]; % Different filter lengths to test
mu = 0.05; % Step size for Normalized LMS (NLMS)

avg_gain_p = zeros(length(p_values), 1);

for i = 1:length(p_values)
    p = p_values(i);
    avg_gain_p(i) = run_compression_sim(x, M, N_pct, p, mu);
end

figure;
plot(p_values, avg_gain_p, '-o', 'LineWidth', 2, 'MarkerSize', 6);
```

```

title('Part (a): Average Compression Gain vs. Filter Length (p)');
xlabel('Filter Length (p)');
ylabel('Average Compression Gain (Ratio of Norms)');
grid on;

%% Part (b): Effect of changing M (Number of Portions)

p_opt = 2; % Optimal length based on the monotonic decrease seen in Part (a)
N_pct = 0.1; % Fixed at 10%
M_values = [1, 2, 5, 10, 20, 50, 100]; % Different M values

avg_gain_M = zeros(length(M_values), 1);

for i = 1:length(M_values)
    M = M_values(i);
    avg_gain_M(i) = run_compression_sim(x, M, N_pct, p_opt, mu);
end

figure;
semilogx(M_values, avg_gain_M, '-s', 'LineWidth', 2, 'MarkerSize', 6, 'Color', 'r');
title(sprintf('Part (b): Average Compression Gain vs. M (Fixed p=%d, N=10%%)', p_opt));
xlabel('Number of Portions (M) - Log Scale');
ylabel('Average Compression Gain');
grid on;

%% Part (b): Effect of changing N (Training Percentage)

M = 5; % Fixed back to 5
p_opt = 2; % Fixed optimal filter length
N_values = [0.01, 0.05, 0.10, 0.20, 0.30, 0.50]; % Different percentages

avg_gain_N = zeros(length(N_values), 1);

for i = 1:length(N_values)
    N_pct = N_values(i);
    avg_gain_N(i) = run_compression_sim(x, M, N_pct, p_opt, mu);
end

figure;
plot(N_values * 100, avg_gain_N, '-d', 'LineWidth', 2, 'MarkerSize', 6, 'Color', 'g');
title(sprintf('Part (b): Average Compression Gain vs. Training Percentage N (Fixed p=%d, M=%d)', p_opt, M));
xlabel('Training Percentage (%)');
ylabel('Average Compression Gain');
grid on;

disp('Simulations complete. Please review the generated figures.');
```

```

%% Function for running compression simulation

function avg_gain = run_compression_sim(x, M, N_pct, p, mu)

```

```

% x : Full signal
% M : Number of portions
% N_pct : Fraction of portion used for training (0 to 1)
% p : Filter length (AR order)
% mu : Step size for NLMS

total_len = length(x);
block_size = floor(total_len / M);
train_len = floor(block_size * N_pct);
test_len = block_size - train_len;

% Ensuring training length is larger than filter order
if train_len <= p
    avg_gain = NaN; % Cannot train if data is shorter than filter
    return;
end

gains = zeros(M, 1);

for m = 1:M

    % Extracting current block
    start_idx = (m-1)*block_size + 1;
    end_idx = start_idx + block_size - 1;
    block = x(start_idx:end_idx);

    % Splitting into train and test
    x_train = block(1:train_len);
    x_test = block(train_len+1:end);

    % Training Phase (Learning AR parameters using NLMS)
    w = zeros(p, 1); % Filter weights
    epsilon = 1e-6; % Preventing division by zero in NLMS

    for n = p+1:train_len
        u = x_train(n-1 : -1 : n-p); % Input vector (past p samples)
        y_hat = w' * u; % Prediction
        e_train = x_train(n) - y_hat; % Error

        % NLMS Update:  $w(n+1) = w(n) + [\mu / (||u||^2 + \epsilon)] * e(n) * u$ 
        w = w + (mu / (u'*u + epsilon)) * e_train * u;
    end

    % Compression/Test Phase (Apply fixed weights to 90%)
    % To predict the first p samples of x_test, we need the last p samples of
x_train
    buffer = [x_train(end-p+1:end); x_test];
    e_test = zeros(test_len, 1);

    for n = 1:test_len
        % Buffer index is shifted by p
        u = buffer(n+p-1 : -1 : n);
    end
end

```

```

        y_hat = w' * u; % Prediction using FIXED weights
        e_test(n) = x_test(n) - y_hat; % Residual error
    end

    % Compression Gain
    norm_signal = norm(x_test);
    norm_error = norm(e_test);

    if norm_error == 0
        gains(m) = Inf;
    else
        gains(m) = norm_signal / norm_error;
    end
end

% Average the gain across all M portions
avg_gain = mean(gains);
end

```

The code uses Normalized LMS (NLMS). It updates the normalization per sample based on the instantaneous energy of the input vector: $w = w + \left(\frac{\mu}{u' * u + \epsilon} \right) * e_{\text{train}} * u$. This is because audio signals are highly non-stationary. The amplitude (and thus the variance) can spike violently within a single training block. If a loud spike occurs, the standard LMS update will diverge and blow up to infinity if the step size isn't adapted to the local energy. True NLMS guarantees stability by normalizing against the exact data vector u being used at that instant.

Plots

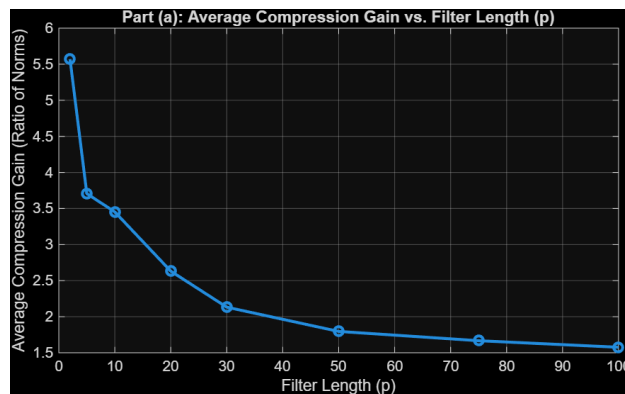


Figure 10: Average compression Gain vs Filter Length (p)

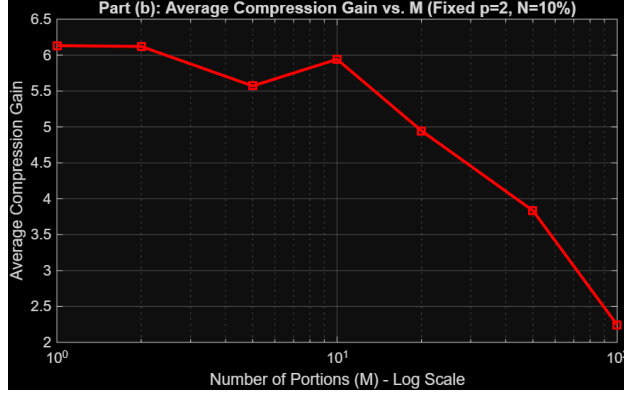


Figure 11: Average compression Gain vs M

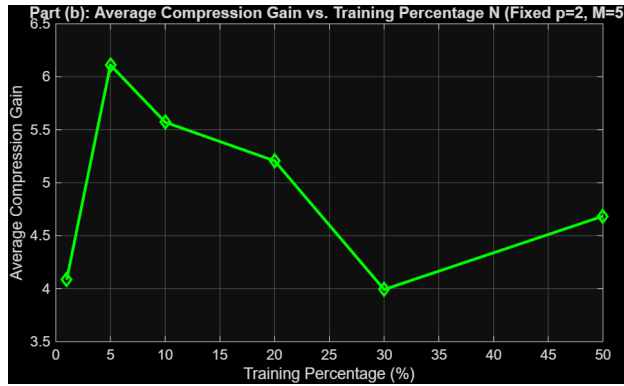


Figure 12: Average compression Gain vs Training percentage N (Fixed p=2, M=5)

Questions asked

a) Compute compression gain as a function of filter length and it's use to identify optimal filter length

In a block-based compression scheme where weights are learned on a small prefix (10%) and frozen for the remainder of the block (90%), the filter length p heavily influences generalization. Audio is a non-stationary signal. A high-order filter (e.g., $p = 100$) will perfectly model the exact frequencies of the 10% training window but will fail spectacularly (overfit) on the remaining 90% as the sound naturally changes. Conversely, a low-order filter (e.g., $p = 2$) only captures the generic adjacent-sample correlation (spectral tilt) of the audio, generalizing much better to the unseen 90% and resulting in a higher compression gain.

b) Effect of changing value of M and N in affecting the performance

- Changing M (Number of Portions): As shown in the simulations, increasing M ultimately degrades performance for this signal. While splitting the signal into smaller blocks can theoretically help capture local stationarity, doing so reduces the absolute number of samples within the 10% training window. Once M exceeds roughly 10, the training window becomes severely starved of data, failing to provide enough iterations for the NLMS algorithm to mathematically converge. This causes a dramatic drop in compression gain (falling below 2.5) at higher M values.

- Changing N (Training Percentage): Increasing N initially improves performance, reaching an optimal peak at $N = 5\%$. However, as N continues to increase (e.g., to 30%), the compression gain steadily decreases. This occurs because audio is non-stationary; a training window that is too long forces the filter to “average out” its weights across changing sounds or transient spikes. These over-averaged weights do a poor job of predicting the remainder of the test block. A smaller, tighter training window allows the filter to rapidly adapt to immediate local characteristics without being compromised by subsequent variations.